C programming……

Q1. What is the output of the following code?

```
int a = 5, b = 2;
printf("%d", a / b * b);
```

Options:

a) 5

b) 4 ✅

c) 2

d) 1

Explanation:

Integer division → a / b = 2 (not 2.5)

So 2 * b = 4.

🔹 **Q2. What is the output of the following program?**

```
int x = 10;s
int *p = &x;
printf("%d", ++*p);
```

Answer: 11 ✅

Explanation:

++*p means increment the value pointed by p.

Equivalent to x = x + 1.

🔹 **Q3. What will be the output?**

```
int a = 10;
printf("%d %d %d", a, ++a, a++);
```

Answer: Undefined Behavior ✅

Explanation:

a is modified more than once between sequence points → output is compiler dependent.

(May print 12 12 10 or 12 11 11, etc.)

🔹 **Q4. Predict the output:**

```
int x = 5;
if (x = 0)
```

printf("Zero");

else

printf("Non-Zero");

Answer: Non-Zero ✅

Explanation:

In C, if (x = 0) assigns 0 to x (not compares).

Condition becomes false (0), so else block executes → "Non-Zero".

🔹 **Q5. What is the output?**

```
int a = 3;
switch(a)
{
default: printf("BEL");
case 1: printf("One");
case 2: printf("Two");
}
```

Answer: BELOneTwo ✅

Explanation:

No break statements → *fall-through* behavior.

🔹 **Q6. Which statement is true about pointers?**

a) A pointer stores a value.

b) A pointer stores an address. ✅

c) A pointer stores a structure.

d) A pointer cannot be used for arrays.

Explanation:

Pointers store memory addresses of variables or arrays.

🔹 **Q7. What is the output?**

```
int a[5] = {1, 2, 3, 4, 5};
printf("%d", *(a + 3));
```

Answer: 4 ✅

Explanation:

a is a pointer to the first element.

*(a + 3) → value at 4th element.

🔹 **Q8. Output of the following code:**

```
char s[] = "BEL";
printf("%d", sizeof(s));
```

Answer: 4 ✅

Explanation:

"BEL" includes an implicit '\0' (null terminator).

Hence, size = 4 bytes.

🔹 **Q9. What is the output of the following code?**

```
int x = 0;
while(x < 3)
{
printf("%d ", x);
x++;
}
```

Answer: 0 1 2 ✅

Explanation:

While loop runs until x < 3 → prints 0, 1, 2.

🔹 **Q10. Which of the following is NOT a valid storage class in C?**

a) auto

b) static

c) register

d) public ✅

Explanation:

public is a C++ keyword, not valid in C.

🔹 **Q11. What is the output?**

```
int arr[] = {10, 20, 30, 40};
int *ptr = arr;
```

ptr += 2;

printf("%d", *ptr);

Answer: 30 ✅

Pointer moved two positions ahead.

🔹 **Q12. Output of the following:**

```
int i;
for (i = 0; i < 5; i++);
printf("%d", i);
```

Answer: 5 ✅

Explanation:

The semicolon after for ends the loop immediately.

After loop ends, i = 5.

🔹 **Q13. What is printed?**

```
int a = 10;
printf("%d", sizeof(++a));
```

Answer: 4 (or size of int) ✅

Explanation:

sizeof is evaluated at compile time → a is not incremented.

🔹 **Q14. Output of this function call:**

```
void fun(int *p){
*p = *p + 10;
}
int main(){
int x = 5;
fun(&x);
printf("%d", x);
}
```

Answer: 15 ✅

Explanation:

***p modifies the value of x via pointer.**

🔹 **Q15. Which of the following functions opens a file for reading in binary mode?**

a) fopen("file.txt", "r")

b) fopen("file.txt", "rb") ✅

c) fopen("file.txt", "w")

d) fopen("file.txt", "wb")

Explanation:

rb → *read binary mode*.

🔹 **Q16. What is the output?**

```
int f(int n){
if(n == 0) return 0;
return n + f(n-1);
}
int main(){
printf("%d", f(4));
}
```

Answer: 10 ✅

Explanation:

f(4) = 4 + 3 + 2 + 1 + 0 = 10 (sum of first 4 integers).

🔹 **Q17. Which of the following is true for C language?**

a) C is a high-level object-oriented language

b) C is a low-level assembly language

c) C is a middle-level structured language ✅

d) C is an interpreted language

Explanation:

C combines low-level features (pointers, memory access) with high-level constructs (functions, loops).

🔹 **Q18. Find the output:**

```
int x = 10;
printf("%d", (x++, ++x));
```

Answer: Compiler Error / Undefined Behavior ✅

Because x is modified twice before a sequence point.

- **Q19. What will be the output?**

char c = 'A';

printf("%d", c);

Answer: 65 ✅

Explanation:

Characters are stored as ASCII values in C.

'A' → 65.

- **Q20. Which header file is needed for malloc() function?**

a) <stdio.h>

b) <conio.h>

c) <stdlib.h> ✅

d) <math.h>

Explanation:

Dynamic memory functions like malloc(), calloc(), and free() are defined in <stdlib.h>.

Data Structures

ARRAYS

- **Definition**

An array is a collection of elements of the same data type stored in contiguous memory.

- **Syntax**

```
int arr[5] = {10, 20, 30, 40, 50};
```

- **Key Operations**
  - **Access: O(1)**
  - **Insertion/Deletion (middle): O(n)**
  - **Traversal: O(n)**
- **Example:**

```
for(int i=0; i<5; i++)
printf("%d ", arr[i]);
```

Output: 10 20 30 40 50

2️⃣ LINKED LIST

🔹 Definition

A Linked List is a collection of nodes, where each node contains:

1. **Data**
2. **Pointer to next node**

🔹 Node Structure

```
struct Node {
int data;
struct Node *next;
};
```

🔹 Advantages

- **Dynamic size**
- **Easy insertion/deletion**

🔹 Disadvantages

- **No random access**
- **Extra pointer memory**

🔹 Example:

```
struct Node a = {10, NULL};
struct Node b = {20, NULL};
a.next = &b;
printf("%d", a.next->data); // Output: 20
```

3️⃣ STACK

🔹 **Definition**

A Stack is a LIFO (Last In, First Out) data structure.

🔹 **Operations**

- push() → Insert at top
- pop() → Remove from top
- peek() → Access top element

🔹 **Implementation**

```
#define MAX 100
int stack[MAX], top = -1;

void push(int val){ stack[++top] = val; }
int pop(){ return stack[top--]; }
```

🔹 **Example**

push(10)

push(20)

pop() → 20

🔹 **Applications**

- Recursion (function call stack)
- Expression evaluation
- Undo operations

4️⃣ QUEUE

🔹 **Definition**

A Queue follows FIFO (First In, First Out) principle.

🔹 **Operations**

- enqueue() – Insert at rear
- dequeue() – Remove from front

🔹 Implementation

```
#define MAX 100
int queue[MAX], front = 0, rear = -1;

void enqueue(int val){ queue[++rear] = val; }
int dequeue(){ return queue[front++]; }
```

🔹 Applications

- CPU scheduling
- Printer queue
- BFS in Graph

5️⃣ CIRCULAR QUEUE

- Avoids wastage of space in simple queue.
- When rear reaches end, it wraps to beginning (rear = 0).

6️⃣ TREES

🔹 Definition

A Tree is a hierarchical structure of nodes.

🔹 Binary Tree Terms

- Root: Top node
- Leaf: Node with no children
- Height: Longest path from root to leaf

🔹 Binary Search Tree (BST)

- Left subtree < Root < Right subtree

- Traversals

Type	Order	Example
Inorder	L Root R	Sorted order in BST
Preorder	Root L R	Used for copying
Postorder	L R Root	Used for deletion

- Example:

10

/ \

5 15

Inorder → 5 10 15

---

7 GRAPHS

- Definition

A Graph consists of vertices (nodes) and edges (connections).

- Types
  - Directed / Undirected
  - Weighted / Unweighted
- Representation
  - Adjacency Matrix
  - Adjacency List
- Traversals
  - BFS (Breadth-First Search) → Uses Queue
  - DFS (Depth-First Search) → Uses Stack/Recursion

---

8 HASHING

- Definition

Stores data using a key-value mapping.

- Hash Function

Converts a key into an index in the hash table.

- Example:

Key = “BEL”

Hash = H(Key) = 3

- Collision Handling
  - Separate Chaining (Linked Lists)
  - Open Addressing (Linear/Quadratic Probing)

---

9 HEAP

- Definition

A Complete Binary Tree that satisfies the Heap Property:

- Max-Heap: Parent ≥ children
- Min-Heap: Parent ≤ children

- Applications
  - Priority Queue
  - Heap Sort
  - Scheduling algorithms

---

10 TIME COMPLEXITY QUICK REFERENCE

Operation	Array	Linked List	Stack	Queue	BST (avg)
Access	O(1)	O(n)	O(n)	O(n)	O(log n)
Insertion	O(n)	O(1)	O(1)	O(1)	O(log n)
Deletion	O(n)	O(1)	O(1)	O(1)	O(log n)

Operation	Array	Linked List	Stack	Queue	BST (avg)
Search	O(n)	O(n)	O(n)	O(n)	O(log n)

🎯 **BEL Important Questions with Answers**

🧩 **Q1. Which data structure uses LIFO order?**

Answer: Stack

🧩 **Q2. Which data structure uses FIFO order?**

Answer: Queue

🧩 **Q3. Which traversal gives sorted output in BST?**

Answer: Inorder Traversal

🧩 **Q4. Which data structure is used for recursion?**

Answer: Stack

🧩 **Q5. What is the time complexity of binary search?**

Answer: O(log n)

🧩 **Q6. What is the space complexity of recursion?**

Answer: O(n) (due to call stack)

🧩 **Q7. Which data structure is used in BFS traversal of graph?**

Answer: Queue

🧩 **Q8. Write output:**

```
int a[5] = {10, 20, 30, 40, 50};
printf("%d", *(a+3));
```

Answer: 40

🧩 **Q9. Write output:**

```
struct Node { int data; struct Node* next; };
struct Node n1 = {10, NULL};
struct Node n2 = {20, NULL};
n1.next = &n2;
printf("%d", n1.next->data);
```

Answer: 20

🧩 **Q10. What is the worst-case time complexity of Quick Sort?**

Answer: $O(n^2)$

(Best/Average: O(n log n))

🧩 **Q11. Which data structure is used in Dijkstra's shortest path algorithm?**

Answer: Priority Queue (Min-Heap)

🧩 **Q12. What is the number of edges in a complete graph of n vertices?**

Answer: n*(n-1)/2

🧩 **Q13. Which searching algorithm is best for sorted arrays?**

Answer: Binary Search

🧩 **Q14. Difference between Stack and Queue?**

Stack	Queue
LIFO	FIFO
push/pop	enqueue/dequeue
Used in recursion	Used in scheduling

🧩 **Q15. What is Hash Collision?**

Answer:

When two keys map to the same hash index.

Handled by:

1. Chaining
2. Open Addressing

🧩 **Q16. Which traversal uses recursion by default?**

Answer: DFS (Depth First Search)

🧩 **Q17. What is the height of a complete binary tree with n nodes?**

Answer: O(log n)

🧩 **Q18. What are applications of Queue?**

- CPU scheduling
- Printer queue
- BFS traversal

🧩 **Q19. What are applications of Stack?**

- Recursion

- Undo operations
- Expression parsing

🧩 Q20. Which sorting algorithms are in-place?

Answer: Bubble, Insertion, Selection, Quick sort

✅ BEL Key Focus Areas

- Stack & Queue operations
- Binary Search Tree properties
- Time complexity (Big O)
- Sorting & Searching
- Graph BFS/DFS
- Hashing & Collision handling

Would you like me to continue with Sorting & Searching (concepts + BEL-style MCQs with solutions) next?

That's the next high-weightage topic in BEL CSE exams.

You said:

Algorithms type question and concepts

ChatGPT said:

Excellent 👏 — this is the right next step for your BEL Trainee Engineer (CSE) written test.

Algorithms form ~25–30% of the technical section along with C and Data Structures.

Let's go through this in the BEL exam-focused way:

- ✅ Core Algorithm Concepts (clear and short)
- ✅ Types of Algorithms (classification)

- ✅ Important BEL-style Questions (with answers and explanations)

⚙️ ALGORITHMS – Concepts & Important Questions

🧩 1️⃣ What is an Algorithm?

An algorithm is a finite set of instructions that performs a specific task or solves a problem.

Example:

Step 1: Start

Step 2: Input a, b

Step 3: c = a + b

Step 4: Print c

Step 5: Stop

🧠 2️⃣ Characteristics of a Good Algorithm

Feature	Description
Finiteness	Must end after a finite number of steps
Definiteness	Each step should be clear
Input	Zero or more inputs
Output	At least one output
Effectiveness	Should be simple and efficient

🔹 3️⃣ Time and Space Complexity

Term	Meaning
Time Complexity	Amount of time an algorithm takes

Term	Meaning
Space Complexity	Memory used during execution

Common Time Complexities

Complexity	Example
O(1)	Accessing array element
O(log n)	Binary Search
O(n)	Linear Search
O(n log n)	Merge/Quick Sort
$O(n^2)$	Bubble/Insertion Sort

🔹 4️⃣ **Asymptotic Notations**

Notation	Meaning
O(f(n))	Upper bound (worst case)
Ω(f(n))	Lower bound (best case)
Θ(f(n))	Average or tight bound

Example:

If an algorithm takes $3n^2 + 5n + 1$ steps →

Time complexity: $O(n^2)$

⚡ 5️⃣ **Types of Algorithms**

Let's divide them by strategy 👇

🟦 **1. Brute Force Algorithm**

- Tries all possible solutions.

- Simple but inefficient for large inputs.

🧩 Example:

Linear Search → check every element one by one.

🟩 2. Divide and Conquer

- Divide the problem into smaller parts
- Conquer each part recursively
- Combine results

🧩 Examples:

- Merge Sort
- Quick Sort
- Binary Search

🟥 3. Greedy Algorithm

- Makes the locally optimal choice at each step.
- Works for optimization problems.

🧩 Examples:

- Dijkstra's Shortest Path
- Kruskal's and Prim's MST
- Huffman Coding

🟧 4. Dynamic Programming (DP)

- Solves problems by breaking into subproblems and storing results (avoid recomputation).

🧩 Examples:

- Fibonacci series (using memoization)

- 0/1 Knapsack
- Matrix Chain Multiplication

🟥 5. Backtracking

- Tries to build solution step-by-step, backtracks when invalid.
- Used for combinatorial problems.

🧩 Examples:

- N-Queens problem
- Sudoku Solver
- Maze path finding

🟪 6. Graph Algorithms

- Solve problems involving nodes and edges.

🧩 Examples:

- BFS (Queue-based)
- DFS (Stack/Recursion-based)
- Dijkstra's (Shortest Path)
- Kruskal's / Prim's (Minimum Spanning Tree)

🟫 7. Searching Algorithms

Algorithm	Type	Time Complexity
Linear Search	Sequential	O(n)
Binary Search	Divide & Conquer	O(log n)

Binary Search Example:

```
int binarySearch(int arr[], int n, int key){
int low=0, high=n-1, mid;
```

```
while(low <= high){
mid = (low + high)/2;
if(arr[mid] == key) return mid;
else if(arr[mid] < key) low = mid + 1;
else high = mid - 1;
}
return -1;
}
```

🔷 8. Sorting Algorithms

Algorithm	Approach	Best	Average	Worst	Stable
Bubble Sort	Compare & Swap	O(n)	$O(n^2)$	$O(n^2)$	✅
Insertion Sort	Shift & Insert	O(n)	$O(n^2)$	$O(n^2)$	✅
Selection Sort	Min selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	❌
Merge Sort	Divide & Conquer	O(n log n)	O(n log n)	O(n log n)	✅
Quick Sort	Partition	O(n log n)	O(n log n)	$O(n^2)$	❌
Heap Sort	Binary Heap	O(n log n)	O(n log n)	O(n log n)	❌

💡 🚶 Graph Traversal Examples

🔸 BFS (Breadth-First Search)

- Uses Queue
- Visits all nodes level by level

Example Use: Finding shortest path in an unweighted graph

🔸 DFS (Depth-First Search)

- Uses Stack or Recursion

- Visits nodes as deep as possible first

Example Use: Detecting cycles in graph

🎯 BEL Algorithm Questions & Solutions

🧩 Q1. What is the time complexity of binary search?

✅ O(log n)

🧩 Q2. Which algorithm design technique is used by Merge Sort?

✅ Divide and Conquer

🧩 Q3. What is the best case time complexity of Quick Sort?

✅ O(n log n)

🧩 Q4. Which algorithm uses dynamic programming?

✅ Fibonacci using memoization / 0-1 Knapsack

🧩 Q5. Which algorithm is used to find shortest path in weighted graph?

✅ Dijkstra's Algorithm

🧩 Q6. Which algorithm is used to find Minimum Spanning Tree?

✅ Prim's and Kruskal's Algorithms

🧩 Q7. What is the main difference between BFS and DFS?

BFS	DFS
Uses Queue	Uses Stack/Recursion
Level order traversal	Depth-wise traversal
Shortest path	Path existence

🧩 **Q8. Output of the following code:**

```
int fun(int n){
if(n==0) return 0;
return n + fun(n-1);
}
int main(){
printf("%d", fun(4));
}
```

Answer: 10 (Sum of first 4 numbers)

🧩 **Q9. Which sorting algorithm is best for large datasets?**

✅ **Merge Sort / Heap Sort / Quick Sort**

🧩 **Q10. What is the space complexity of recursive binary search?**

✅ **O(log n) (due to recursion stack)**

🧩 **Q11. What is the main difference between Greedy and Dynamic Programming?**

Greedy	Dynamic Programming
Local optimization	Global optimization

Greedy	Dynamic Programming
No overlapping subproblems	Overlapping subproblems
Example: Dijkstra	Example: Knapsack

🧩 **Q12. Which algorithm is best for finding all shortest paths between all pairs of vertices?**

✅ **Floyd–Warshall Algorithm**

🧩 **Q13. What is the worst case time complexity of Merge Sort?**

✅ **O(n log n)**

🧩 **Q14. Which sorting algorithm is stable?**

✅ **Bubble, Insertion, Merge Sort**

🧩 **Q15. Which algorithm is used to solve N-Queens problem?**

✅ **Backtracking**

🧩 **Q16. What is the time complexity of BFS/DFS traversal?**

✅ **O(V + E) (V = vertices, E = edges)**

🧩 **Q17. Which algorithm finds Minimum Spanning Tree using edge sorting?**

✅ **Kruskal's Algorithm**

🧩 **Q18. Write the recurrence relation for Merge Sort.**

✅ T(n) = 2T(n/2) + O(n)

(Solved using Master's theorem → O(n log n))

🧩 Q19. Which algorithm is best for detecting a cycle in a graph?

✅ DFS

🧩 Q20. Which searching algorithm works only on sorted data?

✅ Binary Search

- ✅ Core Algorithm Concepts (clear and short)
- ✅ Types of Algorithms (classification)
- ✅ Important BEL-style Questions (with answers and explanations)

⚙️ ALGORITHMS – Concepts & Important Questions

🧩 1️⃣ What is an Algorithm?

An algorithm is a finite set of instructions that performs a specific task or solves a problem.

Example:

Step 1: Start

Step 2: Input a, b

Step 3: c = a + b

Step 4: Print c

Step 5: Stop

🧠 2 Characteristics of a Good Algorithm

Feature	Description
Finiteness	Must end after a finite number of steps
Definiteness	Each step should be clear
Input	Zero or more inputs
Output	At least one output
Effectiveness	Should be simple and efficient

🔹 3 Time and Space Complexity

Term	Meaning
Time Complexity	Amount of time an algorithm takes
Space Complexity	Memory used during execution

Common Time Complexities

Complexity	Example
O(1)	Accessing array element
O(log n)	Binary Search
O(n)	Linear Search
O(n log n)	Merge/Quick Sort
$O(n^2)$	Bubble/Insertion Sort

🔹 4 Asymptotic Notations

Notation	Meaning
O(f(n))	Upper bound (worst case)

Notation	Meaning
Ω(f(n))	Lower bound (best case)
Θ(f(n))	Average or tight bound

Example:

If an algorithm takes $3n^2 + 5n + 1$ steps →

Time complexity: $O(n^2)$

⚡ 5️⃣ Types of Algorithms

Let's divide them by strategy 👇

🟦 1. Brute Force Algorithm

- **Tries all possible solutions.**
- **Simple but inefficient for large inputs.**

🧩 Example:

Linear Search → check every element one by one.

🟩 2. Divide and Conquer

- **Divide the problem into smaller parts**
- **Conquer each part recursively**
- **Combine results**

🧩 Examples:

- **Merge Sort**
- **Quick Sort**
- **Binary Search**

🟧 3. Greedy Algorithm

- Makes the locally optimal choice at each step.
- Works for optimization problems.

🧩 Examples:

- Dijkstra's Shortest Path
- Kruskal's and Prim's MST
- Huffman Coding

🟧 4. Dynamic Programming (DP)

- Solves problems by breaking into subproblems and storing results (avoid recomputation).

🧩 Examples:

- Fibonacci series (using memoization)
- 0/1 Knapsack
- Matrix Chain Multiplication

🟥 5. Backtracking

- Tries to build solution step-by-step, backtracks when invalid.
- Used for combinatorial problems.

🧩 Examples:

- N-Queens problem
- Sudoku Solver
- Maze path finding

🟪 6. Graph Algorithms

- Solve problems involving nodes and edges.

🧩 Examples:

- **BFS (Queue-based)**
- **DFS (Stack/Recursion-based)**
- **Dijkstra's (Shortest Path)**
- **Kruskal's / Prim's (Minimum Spanning Tree)**

🟫 7. Searching Algorithms

Algorithm	Type	Time Complexity
Linear Search	Sequential	O(n)
Binary Search	Divide & Conquer	O(log n)

Binary Search Example:

```
int binarySearch(int arr[], int n, int key){
int low=0, high=n-1, mid;
while(low <= high){
mid = (low + high)/2;
if(arr[mid] == key) return mid;
else if(arr[mid] < key) low = mid + 1;
else high = mid - 1;
}
return -1;
}
```

🔷 8. Sorting Algorithms

Algorithm	Approach	Best	Average	Worst	Stable
Bubble Sort	Compare & Swap	O(n)	$O(n^2)$	$O(n^2)$	✅
Insertion Sort	Shift & Insert	O(n)	$O(n^2)$	$O(n^2)$	✅

Algorithm	Approach	Best	Average	Worst	Stable
Selection Sort	Min selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	❌
Merge Sort	Divide & Conquer	O(n log n)	O(n log n)	O(n log n)	✅
Quick Sort	Partition	O(n log n)	O(n log n)	$O(n^2)$	❌
Heap Sort	Binary Heap	O(n log n)	O(n log n)	O(n log n)	❌

💡 9️⃣ Graph Traversal Examples

🔸 BFS (Breadth-First Search)

- Uses Queue
- Visits all nodes level by level

Example Use: Finding shortest path in an unweighted graph

🔸 DFS (Depth-First Search)

- Uses Stack or Recursion
- Visits nodes as deep as possible first

Example Use: Detecting cycles in graph

🎯 BEL Algorithm Questions & Solutions

🧩 Q1. What is the time complexity of binary search?

✅ O(log n)

🧩 Q2. Which algorithm design technique is used by Merge Sort?

✅ Divide and Conquer

🧩 Q3. What is the best case time complexity of Quick Sort?

✅ O(n log n)

🧩 Q4. Which algorithm uses dynamic programming?

✅ Fibonacci using memoization / 0-1 Knapsack

🧩 Q5. Which algorithm is used to find shortest path in weighted graph?

✅ Dijkstra's Algorithm

🧩 Q6. Which algorithm is used to find Minimum Spanning Tree?

✅ Prim's and Kruskal's Algorithms

🧩 Q7. What is the main difference between BFS and DFS?

BFS	DFS
Uses Queue	Uses Stack/Recursion
Level order traversal	Depth-wise traversal
Shortest path	Path existence

🧩 Q8. Output of the following code:

```
int fun(int n){
if(n==0) return 0;
return n + fun(n-1);
}
int main(){
printf("%d", fun(4));
}
```

Answer: 10 (Sum of first 4 numbers)

🧩 **Q9. Which sorting algorithm is best for large datasets?**

✅ **Merge Sort / Heap Sort / Quick Sort**

🧩 **Q10. What is the space complexity of recursive binary search?**

✅ **O(log n) (due to recursion stack)**

🧩 **Q11. What is the main difference between Greedy and Dynamic Programming?**

Greedy	Dynamic Programming
Local optimization	Global optimization
No overlapping subproblems	Overlapping subproblems
Example: Dijkstra	Example: Knapsack

🧩 **Q12. Which algorithm is best for finding all shortest paths between all pairs of vertices?**

✅ **Floyd–Warshall Algorithm**

🧩 **Q13. What is the worst case time complexity of Merge Sort?**

✅ **O(n log n)**

🧩 **Q14. Which sorting algorithm is stable?**

✅ **Bubble, Insertion, Merge Sort**

🧩 **Q15. Which algorithm is used to solve N-Queens problem?**

✅ **Backtracking**

🧩 **Q16. What is the time complexity of BFS/DFS traversal?**

✅ **O(V + E) (V = vertices, E = edges)**

🧩 **Q17. Which algorithm finds Minimum Spanning Tree using edge sorting?**

✅ **Kruskal's Algorithm**

🧩 **Q18. Write the recurrence relation for Merge Sort.**

✅ **T(n) = 2T(n/2) + O(n)**

(Solved using Master's theorem → O(n log n))

🧩 **Q19. Which algorithm is best for detecting a cycle in a graph?**

✅ **DFS**

🧩 **Q20. Which searching algorithm works only on sorted data?**

✅ **Binary Search**

Operating System

What is an Operating System?

An Operating System (OS) is a system software that acts as an interface between the user and hardware.

It manages:

- **CPU**
- **Memory**

- I/O devices
- Files
- System resources

Examples: Windows, Linux, macOS, Android, UNIX

🧩 2️⃣ Functions of Operating System

Function	Description
Process Management	Handles creation, scheduling, and termination of processes
Memory Management	Allocates and deallocates memory
File Management	Manages files and directories
I/O Management	Controls device communication
Security & Protection	Prevents unauthorized access
Error Detection	Detects and handles errors

⚙️ 3️⃣ Process Management Concepts

🔸 Process

A program in execution.

🔸 Process States

1. New
2. Ready
3. Running
4. Waiting (Blocked)
5. Terminated

🔸 Process Control Block (PCB)

Stores process information:

- **Process ID**
- **Registers**
- **Program counter**
- **Scheduling info**
- **Memory info**

🕐 4️⃣ CPU Scheduling

Scheduling decides which process gets CPU next.

Scheduling Criteria

- **CPU Utilization**
- **Throughput**
- **Turnaround Time**
- **Waiting Time**
- **Response Time**

Scheduling Algorithms

Algorithm	Type	Description
FCFS	Non-preemptive	First come, first served
SJF	Non-preemptive	Shortest Job First
SRTF	Preemptive	Shortest Remaining Time First
RR (Round Robin)	Preemptive	Time quantum based
Priority Scheduling	Both	Based on priority number

💡 BEL-style Question 1

Q: Consider 3 processes with burst times:

P1 = 4 ms, P2 = 3 ms, P3 = 1 ms

Using FCFS, find the average waiting time.

Solution:

Process	Burst Time	Waiting Time
P1	4	0
P2	3	4
P3	1	7

👉 **Average Waiting Time = (0 + 4 + 7) / 3 = 3.67 ms**

💡 **Question 2**

Same processes, SJF Scheduling:

Process	Burst	Waiting
P3	1	0
P2	3	1
P1	4	4

✅ **Average Waiting Time = (0 + 1 + 4)/3 = 1.67 ms**

5️⃣ **Deadlock Concepts**

🔹 **Deadlock**

When two or more processes wait forever for each other to release resources.

Necessary Conditions (Coffman's Conditions)

1. **Mutual Exclusion**
2. **Hold and Wait**
3. **No Preemption**

4. **Circular Wait**

If all 4 exist → deadlock possible.

💡 **BEL-style Question 4**

Q: Which condition is *not* necessary for deadlock?

A. Mutual exclusion

B. Hold and wait

C. Preemption allowed

D. Circular wait

✅ **Answer: C. Preemption allowed**

➡️ **Because "No preemption" is required for deadlock.**

Deadlock Handling Methods

1. **Prevention – eliminate one of the 4 conditions**
2. **Avoidance – use safe states (e.g., Banker's Algorithm)**
3. **Detection & Recovery – detect cycle and recover**

🧠 6️⃣ **Memory Management**

Main Concepts:

- **Contiguous Allocation**
- **Paging**
- **Segmentation**
- **Virtual Memory**
- **Page Replacement**

7️⃣ **File Systems**

Concept	Description
File	Collection of data
Directory	Collection of files
Access methods	Sequential, Direct, Indexed
Allocation	Contiguous, Linked, Indexed

💡 BEL-style Question 7

Q: Which allocation method suffers from external fragmentation?

✅ Answer: Contiguous allocation

🧰 8️⃣ Synchronization & Semaphores

🔹 Critical Section Problem

When multiple processes access shared data → must avoid conflicts.

🔹 Semaphore

Used to control access to shared resources.

```
wait(S);   // acquire
critical_section;
signal(S); // release
```

💡 Question 8

Q: Semaphore value can be

A. Only 0 or 1

B. Any integer value

✅ Answer: B. Any integer value

⚡ 9️⃣ Thrashing

Occurs when the CPU spends more time swapping pages than executing instructions (due to small physical memory).

✅ **Solution: Increase number of frames or use working set model.**

1. Which of the following is not a function of the operating system?

A) Process management

B) Memory management

C) Virus detection

D) File management

✅ **Answer:** C) Virus detection

➡️ OS manages resources; antivirus is a separate application.

🧩 **2. Which of the following best describes a process?**

A) A program in secondary memory

B) A program in execution

C) A job in the queue

D) A compiled code

✅ **Answer:** B) A program in execution

➡️ A process is an active entity with its own state, registers, and memory.

🧩 **3. What does the Process Control Block (PCB) contain?**

A) Only process ID

B) Process state, registers, memory info

C) Input/output devices

D) Compiler information

✅ **Answer:** B

➡️ PCB stores process ID, CPU registers, scheduling info, etc.

🧩 **4. CPU scheduling is the basis of _____.**

A) Multiprogramming

B) Multiprocessing

C) Time-sharing

D) Both A and C

✅ **Answer:** D

➡️ CPU scheduling enables multiprogramming and time-sharing.

🧩 **5. In which scheduling algorithm can starvation occur?**

A) Round Robin

B) FCFS

C) Priority Scheduling

D) Shortest Job First

✅ **Answer:** C & D (can happen in both)

➡️ Low-priority or long jobs may never get CPU.

🧩 **6. Which algorithm has the least average waiting time if all processes arrive at the same time?**

A) FCFS

B) SJF

C) RR

D) Priority

✅ **Answer:** B) SJF

➡️ Shortest Job First minimizes average waiting time.

🧩 **7. What is the average waiting time using FCFS for processes with burst times 3, 5, 2?**

Solution:

Process	Burst	Waiting
P1	3	0
P2	5	3
P3	2	8

✅ **Average = (0+3+8)/3 = 3.67 ms**

🧩 **8. Which algorithm uses time slicing?**

A) FCFS

B) Round Robin

C) SJF

D) Priority

✅ **Answer:** B) Round Robin

🧩 **9. What is a critical section?**

A) Part of code accessing shared resources

B) Kernel area

C) User input area

D) None

✅ **Answer:** A

➡️ Only one process should execute in the critical section at a time.

🧩 **10. What is the main role of a semaphore?**

A) Scheduling

B) Synchronization

C) Memory allocation

D) Device control

✅ **Answer:** B

➡️ Semaphore prevents race conditions in process synchronization.

🧩 **11. Which of the following can cause a deadlock?**

A) Preemption of resources

B) Circular wait

C) Single process execution

D) None

✅ **Answer:** B

➡️ Circular wait among processes is a key cause of deadlocks.

🧩 **12. Which of the following is used to avoid deadlock?**

A) Resource ordering

B) Wait and hold

C) Mutual exclusion

D) Circular wait

✅ **Answer:** A

➡️ Ordering resources avoids circular waiting.

🧩 **13. In paging, the logical address is divided into _____.**

A) Page number and page offset

B) Frame and offset

C) Segment and offset

D) None

✅ **Answer:** A

➡️ Logical Address = Page Number + Offset.

🧩 **14. Which of the following eliminates external fragmentation?**

A) Paging

B) Segmentation

C) Contiguous allocation

D) Swapping

✅ **Answer:** A

➡️ Paging allocates fixed-size blocks → no external fragmentation.

🧩 **15. Which page replacement algorithm is optimal but not practical?**

A) FIFO

B) LRU

C) Optimal

D) Clock

✅ **Answer:** C

➡️ Optimal algorithm needs future knowledge — theoretical best.

🧩 **16. Pages: 1,2,3,4,1,2,5,1,2,3,4,5; Frames = 3 (FIFO)**

Solution:

Faults for 1,2,3,4,1,2,5,1,2,3,4,5 → **9 page faults**

✅ **Answer:** 9

🧩 **17. What is virtual memory?**

A) Memory on disk used as RAM extension

B) Physical RAM

C) Cache memory

D) None

✅ **Answer:** A

➡️ Virtual memory allows programs to exceed physical memory limits.

🧩 **18. What is thrashing?**

A) Page fault rate very high

B) CPU utilization very low

C) Both A and B

D) None

✅ **Answer:** C

➡️ Thrashing occurs when too much paging reduces performance.

🧩 **19. What is a system call?**

A) User program's request to OS

B) Interrupt

C) Device driver

D) API

✅ **Answer:** A

➡️ System calls allow user processes to request OS services.

🧩 **20. Which of the following is true for multitasking OS?**

A) Only one process in memory

B) Multiple processes share CPU

C) No scheduling used

D) Single thread of execution

✅ **Answer:** B

➡️ Multitasking OS executes multiple processes by time-sharing CPU.

Computer Network

A system where multiple computers are connected to share resources (data, printers, internet, etc.).

Types:

- **LAN** – Local Area Network (office, building)
- **MAN** – Metropolitan Area Network (city)
- **WAN** – Wide Area Network (country/worldwide, e.g., Internet)

---

2. Network Topologies

Physical or logical arrangement of nodes.

Topology	Description	Example
Bus	One main cable, all nodes connected	Old LANs
Star	All nodes connected to a central hub	Modern LANs
Ring	Each node connected to next forming a ring	Token ring
Mesh	Every node connected to every other	Internet backbone

---

3. OSI Model (7 Layers)

Layer	Function	Protocols
7. Application	User interface	HTTP, FTP, SMTP
6. Presentation	Encryption, compression	SSL, JPEG
5. Session	Establish, manage sessions	NetBIOS
4. Transport	Reliable delivery	TCP, UDP
3. Network	Routing	IP, ICMP
2. Data Link	Framing, MAC	Ethernet, PPP
1. Physical	Transmission of bits	Cables, Hubs

---

4. TCP/IP Model (4 Layers)

1. Application
2. Transport
3. Internet
4. Network Access

➡️ Practical version of OSI model.

---

5. IP Addressing

- IPv4: 32 bits (e.g., 192.168.1.1)
- IPv6: 128 bits (used for modern networks)

Classes:

Class	Range	Default Mask
A	0–127	255.0.0.0
B	128–191	255.255.0.0
C	192–223	255.255.255.0

---

6. Subnetting Example

IP: 192.168.10.0/26

Subnet mask: 255.255.255.192

→ Block size = 64

→ Subnets: 192.168.10.0, .64, .128, .192

→ Each subnet: 62 usable hosts.

---

7. Protocols

Layer	Protocols
Application	HTTP, FTP, SMTP, DNS
Transport	TCP (reliable), UDP (fast)
Network	IP, ICMP
Data Link	Ethernet, ARP

---

8. Transmission Modes

- **Simplex:** One-way (keyboard → CPU)
- **Half-duplex:** Both ways but not same time (Walkie-talkie)
- **Full-duplex:** Both ways simultaneously (telephone)

---

9. Switching Techniques

Type	Description
Circuit switching	Dedicated path
Packet switching	Data in packets
Message switching	Entire message sent

---

10. Error Detection

- **Parity Bit**
- **Checksum**
- **CRC (Cyclic Redundancy Check)**

🧮 BEL-TYPE QUESTIONS (With Solutions)

---

1. Which layer is responsible for routing packets?

A) Data Link

B) Network

C) Transport

D) Session

✅ **Answer:** B

➡️ Network layer determines the route for packets using IP.

---

2. Which protocol provides reliable delivery of data?

A) TCP

B) UDP

C) IP

D) ICMP

✅ **Answer:** A

➡️ TCP provides sequencing, error control, acknowledgment.

---

3. In OSI model, encryption is done at which layer?

A) Network

B) Presentation

C) Transport

D) Application

✅ **Answer:** B

➡️ Presentation layer handles encryption, decryption, compression.

---

4. HTTP uses which transport layer protocol?

A) UDP

B) TCP

C) ICMP

D) IP

✅ **Answer:** B

➡️ HTTP ensures reliable connection using TCP.

---

5. Which address is used by a router to forward packets?

A) MAC address

B) IP address

C) Port number

D) Physical address

✅ **Answer:** B

➡️ Routers operate on Network Layer → use IP.

---

6. What is the default subnet mask for Class B?

✅ **Answer:** 255.255.0.0

---

7. What is the range of Class C IP address?

✅ **Answer:** 192.0.0.0 – 223.255.255.255

---

8. A packet is sent from source 10.0.0.1 to 192.168.1.1. Which device is required?

✅ **Answer:** Router

➡️ Needed to connect different networks.

---

9. Which of the following is not an application layer protocol?

A) FTP

B) SMTP

C) HTTP

D) IP

✅ **Answer:** D

➡️ IP belongs to the network layer.

---

10. Which layer is responsible for flow control and error control?

✅ **Answer:** Transport Layer

---

11. ARP is used to resolve:

✅ **Answer:** IP address → MAC address

➡️ Address Resolution Protocol maps logical to physical address.

---

12. DNS stands for:

✅ **Answer:** Domain Name System

➡️ Converts domain names into IP addresses.

---

13. Which of the following uses connectionless service?

A) FTP

B) SMTP

C) DNS

D) HTTP

✅ **Answer:** C

➡️ DNS uses UDP → connectionless.

---

14. Which layer adds source and destination port numbers?

✅ **Answer:** Transport Layer

➡️ Ports identify specific processes.

---

15. How many usable hosts are in subnet 255.255.255.192?

Mask bits = /26 → Block size = 64 → Usable = 64 - 2 = **62 hosts**

✅ **Answer:** 62

---

16. Which protocol is used for email transmission?

✅ **Answer:** SMTP (Simple Mail Transfer Protocol)

---

17. Which layer is responsible for physical addressing?

✅ **Answer:** Data Link Layer

---

18. What is the function of a switch?

✅ **Answer:** Connect devices within a LAN and forward frames based on MAC address.

---

19. What is the propagation delay if signal travels 2000 km at 2×10^8 m/s?

Distance = 2000×10^3 m

Speed = 2×10^8 m/s

Time = Distance / Speed = $(2\times10^6)/(2\times10^8)$ = **0.01 s = 10 ms**

✅ **Answer:** 10 milliseconds

---

20. What does ICMP stand for and what is its use?

✅ **Answer:** Internet Control Message Protocol

➡️ Used for **error reporting** and **diagnostics** (e.g., ping).

Concept	Key Facts
OSI Layers	7 layers, remember "All People Seem To Need Data Processing"
TCP vs UDP	TCP = reliable, UDP = fast
IP Addressing	IPv4 (32-bit), IPv6 (128-bit)
Devices	Hub (Layer 1), Switch (L2), Router (L3)
Subnet Mask	Defines network/host portion
Protocols	HTTP, FTP, DNS, SMTP, ARP, ICMP
Error Control	CRC, Checksum
Switching	Circuit, Packet, Message

DBMS

A **Database Management System (DBMS)** is software used to store, retrieve, and manage data efficiently.

Examples: MySQL, Oracle, PostgreSQL, MS SQL Server.

Advantages:

- Reduces redundancy
- Ensures data consistency
- Provides security
- Supports concurrent access

---

2. Schema & Instance

- **Schema:** Structure of database (like blueprint)
- **Instance:** Data at a specific moment in time

---

3. Keys in DBMS

Key Type	Description
Primary Key	Uniquely identifies each record
Candidate Key	All keys that can be primary
Super Key	Set of attributes that uniquely identify a row
Foreign Key	Refers to primary key of another table
Composite Key	Key formed by combining multiple columns

---

4. Normalization

It's the process of **organizing data to reduce redundancy** and improve integrity.

Normal Form	Rule
1NF	No repeating groups
2NF	1NF + no partial dependency
3NF	2NF + no transitive dependency
BCNF	Every determinant is a candidate key

---

5. SQL (Structured Query Language)

Operation	Example
SELECT	SELECT name FROM employee;
INSERT	INSERT INTO employee VALUES(1,'John');
UPDATE	UPDATE employee SET salary=50000 WHERE id=1;
DELETE	DELETE FROM employee WHERE id=1;

---

6. Joins

Type	Description
INNER JOIN	Returns only matching rows
LEFT JOIN	All rows from left + matching from right
RIGHT JOIN	All rows from right + matching from left
FULL JOIN	All rows from both sides
SELF JOIN	Joins table to itself

---

7. Transactions & ACID Properties

- **Atomicity** – All or nothing
- **Consistency** – Must leave DB in a valid state
- **Isolation** – Concurrent transactions do not interfere
- **Durability** – Once committed, data is permanent

---

8. Concurrency Control

Used when multiple transactions execute together.

Techniques:

- Locking (Shared/Exclusive)
- Timestamp ordering

- Two-Phase Locking (2PL)

---

9. Deadlock in DBMS

Occurs when two transactions hold locks and wait for each other.

Can be **prevented** or **resolved by rollback**.

---

10. Relational Algebra (Basic Operations)

- **Select (σ):** Choose rows → σ salary>50000(Employee)
- **Project (π):** Choose columns → π name, dept(Employee)
- **Join (⋈):** Combine related tuples
- **Union (U):** Combine sets
- **Set Difference (-):** Subtract sets

🧮 **BEL-TYPE QUESTIONS (With Solutions)**

---

1. Which of the following is not a property of a transaction?

A) Atomicity

B) Consistency

C) Deadlock

D) Durability

✅ **Answer:** C

➡️ Deadlock is a concurrency problem, not a transaction property.

---

2. Which key uniquely identifies a record in a table?

✅ **Answer:** Primary Key

---

3. Which SQL command is used to remove a table permanently?

✅ **Answer:** DROP TABLE table_name;

---

4. What is the difference between DELETE and TRUNCATE?

DELETE	TRUNCATE
Removes rows one by one	Removes all rows instantly
Can use WHERE	Cannot use WHERE
Slower	Faster

---

5. Find the output:

SELECT COUNT(*) FROM Employee WHERE dept='HR';

✅ **Answer:** Returns the number of employees working in HR department.

---

6. What does a foreign key do?

✅ **Answer:** Enforces referential integrity between two tables.

---

7. Which SQL statement retrieves unique department names?

✅ **Answer:**

SELECT DISTINCT dept FROM Employee;

---

8. Which command is used to modify an existing table structure?

✅ **Answer:**

ALTER TABLE table_name ADD column_name datatype;

---

9. Which normal form removes transitive dependency?

✅ **Answer:** 3NF (Third Normal Form)

---

10. Which of the following represents 1NF?

✅ **Answer:** No repeating groups or multi-valued attributes.

---

11. A table with partial dependency violates which normal form?

✅ **Answer:** 2NF

➡️ Partial dependency = non-prime attribute depends on part of a composite key.

---

12. What is the result of this query?

SELECT name FROM student WHERE marks BETWEEN 60 AND 80;

✅ **Answer:** Returns names of students having marks between 60 and 80.

---

13. What does COMMIT do?

✅ **Answer:** Saves all changes made in the transaction permanently.

---

14. What is the use of ROLLBACK?

✅ **Answer:** Undo changes of a transaction before commit.

---

15. Which of the following ensures isolation in transactions?

A) Locks

B) Views

C) Indexes

D) Constraints

✅ **Answer:** A

➡️ Locks prevent interference between concurrent transactions.

---

16. Which of the following prevents data redundancy?

✅ **Answer:** Normalization

---

17. What is the Cartesian product in relational algebra?

✅ **Answer:** Combines every row of one relation with every row of another.

---

18. Which command is used to rename a table?

✅ **Answer:**

RENAME TABLE old_name TO new_name;

---

19. Which SQL clause is used to group records?

✅ **Answer:** GROUP BY

---

20. What is the difference between INNER JOIN and LEFT JOIN?

INNER JOIN	LEFT JOIN
Returns only matching rows	Returns all left + matching right

✅ **Example:**

```
SELECT e.name, d.dept_name
FROM Employee e
LEFT JOIN Department d ON e.dept_id = d.dept_id;
```

Software Engineering

It is the **systematic, disciplined, and quantifiable approach** to developing, operating, and maintaining software.

➡️ Focuses on **quality, maintainability, and reliability**.

---

2. Software Development Life Cycle (SDLC)

Phases:

1. Requirement Analysis
2. System Design
3. Implementation (Coding)
4. Testing
5. Deployment
6. Maintenance

---

3. SDLC Models

Model	Description	Pros	Cons
Waterfall	Sequential, phase-wise	Simple	Rigid, no feedback
Iterative	Repeats cycles	Early feedback	Costly
Spiral	Combines prototyping & risk analysis	Handles risk	Complex
V-Model	Testing alongside development	Easy validation	Costly
Agile	Incremental, customer-driven	Fast, flexible	Needs communication

---

4. Functional vs Non-functional Requirements

Type	Examples
Functional	Login, report generation, database updates
Non-functional	Performance, security, scalability

---

5. Feasibility Study

Checks if project is practical and profitable.

Types:

- **Technical**
- **Economic**
- **Operational**
- **Schedule feasibility**

---

6. Software Design Principles

- **Modularity:** Divide system into manageable modules
- **Abstraction:** Hide implementation details
- **Cohesion:** Related tasks together (should be high)
- **Coupling:** Dependency between modules (should be low)

---

7. Testing Levels

Level	Purpose
Unit Testing	Tests individual modules
Integration Testing	Tests combined modules
System Testing	Tests complete system
Acceptance Testing	Done by client/end user

---

8. Types of Testing

Type	Description
Black Box	Test input/output only
White Box	Test code structure
Regression Testing	Re-testing after modification

Type	Description
Stress Testing	Check limits of system performance

---

9. Software Metrics

Used to measure attributes like complexity, productivity, quality.

Examples:

- LOC (Lines of Code)
- Cyclomatic Complexity
- Defect Density

---

10. Software Maintenance Types

Type	Description
Corrective	Fix bugs
Adaptive	Change environment (e.g., new OS)
Perfective	Improve performance/features
Preventive	Prevent future problems

📋 BEL-TYPE QUESTIONS (With Solutions)

---

1. Which of the following is not a phase in SDLC?

A) Coding

B) Testing

C) Debugging

D) Design

✅ **Answer:** C

➡️ Debugging is part of coding/testing, not a main SDLC phase.

---

2. In which SDLC model is testing done in parallel with development?

✅ **Answer:** V-Model

---

3. The main aim of requirement analysis is to:

A) Check system performance

B) Understand what the user wants

C) Design the UI

D) Write code

✅ **Answer:** B

➡️ Requirement analysis defines **what** the system should do.

---

4. Spiral model focuses on which key aspect?

✅ **Answer:** Risk Analysis

➡️ Each iteration in spiral model evaluates and reduces project risk.

---

5. Which SDLC model is best suited for well-defined requirements?

✅ **Answer:** Waterfall Model

---

6. What is the main drawback of the Waterfall model?

✅ **Answer:** Difficult to accommodate changes after a phase is complete.

---

7. In Agile model, work is divided into small deliverables called _____.

✅ **Answer:** Iterations or Sprints

---

8. Which of the following is a non-functional requirement?

A) User authentication

B) Response time < 2s

C) Report generation

D) Database update

✅ **Answer:** B

➡️ Non-functional requirements define **quality attributes**.

---

9. What does high cohesion and low coupling mean?

✅ **Answer:** Modules perform specific tasks independently → better maintainability.

---

10. Which design principle focuses on hiding internal details?

✅ **Answer:** Abstraction

---

11. Which testing type checks functionality without seeing code?

✅ **Answer:** Black Box Testing

---

12. What is the goal of integration testing?

✅ **Answer:** To test interfaces between modules.

---

13. Which testing ensures old functionality works after updates?

✅ **Answer:** Regression Testing

---

14. Acceptance testing is done by:

✅ **Answer:** End users or clients

---

15. Which metric measures program complexity?

✅ **Answer:** Cyclomatic Complexity

---

16. Which of these is not a software quality attribute?

A) Reliability

B) Portability

C) Efficiency

D) Debugging

✅ **Answer:** D

---

17. What is the first step in software maintenance?

✅ **Answer:** Problem identification or analysis

---

18. Which maintenance type adapts software to a new environment?

✅ **Answer:** Adaptive Maintenance

---

19. Verification ensures _____, while Validation ensures _____.

✅ **Answer:**

Verification – "Are we building the product right?"

Validation – "Are we building the right product?"

---

20. Which diagram in UML shows data flow between processes?

✅ **Answer:** Data Flow Diagram (DFD)

Computer Organization and Architecture

- **Computer Organization:** How hardware components are connected and interact (e.g., control signals, buses, registers).
- **Computer Architecture:** Design and functionality as seen by a programmer (e.g., instruction set, addressing modes).

➡️ In short:

Architecture = What the computer does

Organization = How it does it

⚙️ 2. Basic Components of a Computer

Component	Function
Input Unit	Accepts data (keyboard, mouse)
Output Unit	Displays results (monitor, printer)
Memory Unit	Stores data/instructions
ALU	Performs arithmetic & logical operations
Control Unit (CU)	Controls operation of all units
Registers	Small, fast storage within CPU

💾 3. Types of Memory

Memory	Description	Speed
Register	Fastest storage inside CPU	🔥 Fastest
Cache	Small, high-speed memory between CPU & RAM	⚡ Fast
Main Memory (RAM)	Stores data/instructions during execution	🧠 Medium

Memory	Description	Speed
Secondary Memory	Hard disk, SSD	🐢 Slow

Memory Hierarchy:

Registers → Cache → RAM → Secondary Storage

🧠 4. Instruction Cycle

Each instruction passes through:

1. **Fetch** (from memory)
2. **Decode** (interpret instruction)
3. **Execute** (perform operation)
4. **Store** (save result)

📜 5. Addressing Modes

Mode	Example	Description
Immediate	MOV A, #5	Operand is constant
Direct	MOV A, [1000]	Address given directly
Indirect	MOV A, [R1]	Address stored in register
Register	MOV A, B	Operand in register
Indexed	MOV A, X + R1	Base + offset

🔁 6. Pipelining

Technique where multiple instructions are overlapped in execution.

If 5-stage pipeline (Fetch, Decode, Execute, Memory, Write Back), ideal speedup = 5×.

✅ Improves throughput,

❌ May suffer from *hazards* (data, control, structural).

🧮 7. RISC vs CISC

Feature	RISC	CISC
Instruction length	Fixed	Variable
Execution speed	Fast	Slower
Example	ARM, MIPS	Intel x86

🔢 8. Number Systems

Type	Base	Example
Binary	2	1011
Octal	8	17
Decimal	10	23
Hexadecimal	16	2A

Conversion Example:

Binary 1010 = Decimal 10

💿 9. Cache Mapping

Type	Description
Direct Mapping	Each block maps to one cache line
Fully Associative	Any block can go anywhere
Set Associative	Combination of both

⚡ 10. Interrupts

Used to handle events that need immediate attention (I/O, timer, errors).

Types:

- **Hardware Interrupt** (Keyboard input)
- **Software Interrupt** (INT instruction)

📋 **BEL-TYPE QUESTIONS with SOLUTIONS**

---

1. Which of the following is not a component of CPU?

A) ALU

B) Control Unit

C) Register

D) Cache

✅ **Answer:** D

➡️ Cache is outside CPU but connected closely.

---

2. Which memory is the fastest?

✅ **Answer:** Register memory

---

3. What is the full form of ALU?

✅ **Answer:** Arithmetic Logic Unit

---

4. Which of the following stores address of next instruction?

✅ **Answer:** Program Counter (PC)

---

5. What are the two main types of memory?

✅ **Answer:** Primary (RAM, ROM) and Secondary (Hard disk)

---

6. Which of the following is a volatile memory?

✅ **Answer:** RAM

➡️ Loses data when power is off.

---

7. In instruction MOV A, #20, which addressing mode is used?

✅ **Answer:** Immediate Addressing

---

8. In instruction MOV A, [3000], which addressing mode is used?

✅ **Answer:** Direct Addressing

---

9. In instruction MOV A, [R1], which addressing mode is used?

✅ **Answer:** Register Indirect Addressing

---

10. Which register holds the address of the current instruction?

✅ **Answer:** Program Counter (PC)

---

11. What is pipelining in CPU?

✅ **Answer:** Overlapping of instruction execution to increase throughput.

---

12. What causes pipeline hazards?

✅ **Answer:** Dependencies between instructions (data or control hazard).

---

13. Which architecture uses few simple instructions?

✅ **Answer:** RISC (Reduced Instruction Set Computer)

---

14. In a 4-stage pipeline, ideal speedup = ?

✅ **Answer:** 4× (if no hazards)

---

15. Which of the following is used to improve CPU performance?

✅ **Answer:** Cache Memory

---

16. Which mapping technique allows any block to be placed anywhere in cache?

✅ **Answer:** Fully Associative Mapping

---

17. The control unit is responsible for:

✅ **Answer:** Directing operations of processor components.

---

18. Convert $(1101)_2$ to decimal.

= 1×8 + 1×4 + 0×2 + 1×1 = **13**

✅ **Answer:** 13

---

19. The hexadecimal number $(2F)_{16}$ equals what in binary?

2 → 0010

F → 1111

✅ **Answer:** 00101111

---

20. The interrupt that cannot be masked or disabled is called?

✅ **Answer:** Non-Maskable Interrupt (NMI)

DIGITAL ELECTRONICS

🧠 Core Concepts

---

1. Number Systems

Type	Base	Example
Binary	2	1010
Octal	8	17
Decimal	10	23
Hexadecimal	16	2F

Conversions:

- Binary → Decimal → Multiply by powers of 2
- Decimal → Binary → Divide by 2 repeatedly

---

2. Boolean Algebra

Basic Laws:

- **Identity:** A + 0 = A, A · 1 = A
- **Complement:** A + A' = 1, A · A' = 0
- **Idempotent:** A + A = A, A · A = A
- **Distributive:** A · (B + C) = A·B + A·C

DeMorgan's Theorems:

1. (A·B)' = A' + B'
2. (A + B)' = A'·B'

---

3. Logic Gates

Gate	Symbol	Expression
AND	·	Y = A·B
OR	+	Y = A+B
NOT	¯	Y = A'
NAND	·'	Y = (A·B)'
NOR	+'	Y = (A+B)'
XOR	⊕	Y = A⊕B
XNOR	⊕'	Y = (A⊕B)'

---

4. Simplification Techniques

- Boolean algebra laws
- Karnaugh Maps (K-map) → 2, 3, 4 variables
- SOP (Sum of Products) / POS (Product of Sums) forms

---

5. Combinational Circuits

- **Adders:** Half adder, Full adder
- **Subtractor**
- **Multiplexer (MUX)**: n-input → 1 output
- **Demultiplexer (DEMUX)**: 1 input → n outputs
- **Encoder / Decoder**

---

6. Sequential Circuits

- Depends on **current state & input**
- **Flip-Flops:** Basic memory elements

Flip-Flop	Triggered By	Characteristic
SR	Level / Edge	Set/Reset
D	Edge	Data
JK	Edge	Toggle
T	Edge	Toggle

- **Registers:** Group of flip-flops
- **Counters:** Sequential flip-flops producing a count sequence

---

7. Timing Concepts

- **Propagation Delay (tpd):** Time for input change → output
- **Setup Time:** Minimum time before clock edge
- **Hold Time:** Minimum time after clock edge

---

8. ADC / DAC

- **ADC (Analog → Digital)**
- **DAC (Digital → Analog)**

📋 **BEL-TYPE QUESTIONS (With Solutions)**

---

1. Convert decimal 25 to binary.

25 ÷ 2 = 12 remainder 1

12 ÷ 2 = 6 remainder 0

6 ÷ 2 = 3 remainder 0

3 ÷ 2 = 1 remainder 1

1 ÷ 2 = 0 remainder 1

✅ **Answer:** 11001_2

---

2. Simplify: A + AB

A + AB = A(1 + B) = A

✅ **Answer:** A

---

3. Simplify using DeMorgan: (A + B)'

✅ **Answer:** A' · B'

---

4. Truth table of XOR (A⊕B)

A B A⊕B

0 0 0

0 1 1

1 0 1

1 1 0

---

5. Which gate is universal?

✅ **Answer:** NAND and NOR

---

6. Half adder outputs

- Sum: A⊕B
- Carry: A·B

---

7. Full adder outputs

- Sum: A⊕B⊕Cin
- Carry: A·B + B·Cin + Cin·A

---

8. Convert 1011_2 to decimal

$1\times8 + 0\times4 + 1\times2 + 1\times1 = 11$

✅ **Answer:** 11

---

9. SOP simplification: F = A·B + A·B'

F = A(B + B') = A·1 = A

✅ **Answer:** F = A

---

10. 2-to-1 MUX selects output using

✅ **Answer:** 1 select line

---

11. 4-bit counter maximum count

$2^4 = 16 \rightarrow$ count from 0 to 15

✅ **Answer:** 15 (decimal)

---

12. D flip-flop characteristic equation

Q(next) = D

---

13. JK flip-flop toggles when

J=1, K=1

---

14. Which circuit stores one bit of data?

✅ **Answer:** Flip-flop

---

15. Convert hex 2F → binary

2 → 0010

F → 1111

✅ **Answer:** 00101111

---

16. Number of minterms in 3-variable Boolean function

2^3 = 8 minterms

---

17. Demux function

1 input → n outputs

---

18. ADC full form

✅ **Answer:** Analog to Digital Converter

---

19. DAC full form

✅ **Answer:** Digital to Analog Converter

---

20. Which of the following is sequential circuit?

A) MUX

B) Encoder

C) Counter

D) Half Adder

✅ **Answer:** C (depends on past state)